Assignment No- 10

Aim: To illustrate the Binary Search Tree.

Problem Statement: A list stores city names and their populations. Use a Binary Search Tree for implementation. Provide a facility for adding new cities, deleting a city, and updating population values. Provide a facility to display all the city names in ascending/descending order. Also, find how many maximum comparisons are required to search for a particular city.

Learning Objectives:

- To understand the concept of Binary Search Trees.
- To apply BST operations for real-world applications.
- To analyze search efficiency using comparison counts.

Learning Outcome: Students will be able to:

- Implement BST operations in Python.
- Manage hierarchical data using BST.
- Compare efficiency of searching with maximum number of comparisons required.

Theory:

- A **Binary Search Tree (BST)** is a special type of binary tree where:
  - The left child contains nodes with keys less than the parent's key.
  - The right child contains nodes with keys greater than the parent's key.
  - Duplicate keys are not allowed.
- A clear advantage with Binary Search Trees is that operations like search, delete, and insert are fast and done without having to shift values in memory.

Operations on BST:

- **Insertion:** Places a new node in the correct position.
- **Deletion:** Removes a node (three cases – leaf node, one child, two children).
- **Searching:** Traverses nodes based on key comparison (left/right).
- **Traversal:**
  - Inorder → Ascending order
  - Reverse Inorder → Descending order

Time Complexity of BST:

- Best case (balanced tree): O(log n)
- Worst case (skewed tree): O(n)
- Maximum comparisons for searching = height of BST (worst case).

EXAMPLE:

We'll insert the keys in this order:
[50, 30, 70, 20, 40, 60, 80]

1) Insertion —

Steps*:*

1. Start at the root node.
2. Compare each node:
   - Is the value lower? Go left.
   - Is the value higher? Go right.
3. Continue to compare nodes with the new value until there is no right or left to compare with. That is where the new node is inserted.

For above example we start with an empty tree.

1. **Insert 50**
   - Tree empty → 50 becomes root.
2. **Insert 30**
   - Compare with 50: 30 < 50 → go left (null) → insert as left child of 50.
3. **Insert 70**
   - Compare with 50: 70 > 50 → go right (null) → insert as right child of 50.
4. **Insert 20**
   - Compare 20 vs 50: 20 < 50 → go left to 30.
   - Compare 20 vs 30: 20 < 30 → go left (null) → insert under 30.
5. **Insert 40**
   - 40 vs 50: 40 < 50 → left to 30.
   - 40 vs 30: 40 > 30 → right (null) → insert under 30.
6. **Insert 60**
   - 60 vs 50: 60 > 50 → right to 70.
   - 60 vs 70: 60 < 70 → left (null) → insert under 70.
7. **Insert 80**
   - 80 vs 50: 80 > 50 → right to 70.
   - 80 vs 70: 80 > 70 → right (null) → insert under 70.

After all inserts, the tree looks like:

```
      50
    /    \
  30      70
 /  \    /  \
20   40  60  80
```

2) Search — (find 60 in BST)

Steps:

1. Start at the root node.
2. If this is the value we are looking for, return.
3. If the value we are looking for is higher, continue searching in the right subtree.
4. If the value we are looking for is lower, continue searching in the left subtree.
5. If the subtree we want to search does not exist, depending on the programming language, return None, or NULL, or something similar, to indicate that the value is not inside the BST.

Goal: Search for **60** and count comparisons.

- **Pass 1:** Compare 60 vs 50 → 60 > 50 → go right to 70.
- **Pass 2:** Compare 60 vs 70 → 60 < 70 → go left to 60.
- **Pass 3:** Compare 60 vs 60 → **found**.

No of Comparisons = 3.

How many maximum comparisons?

- On any BST, **max comparisons = height + 1** (height = edges on longest root→leaf path).
- In a **balanced** BST with n nodes: height ≈ $\lfloor \log_2 n \rfloor$, so **max comparisons ≈ $\lfloor \log_2 n \rfloor + 1$**.
- In the **worst case (skewed)**: height = n−1 → **max comparisons = n**.

For our 7-node tree, $\lfloor \log_2 7 \rfloor = 2$, so a successful search needs at most **3** comparisons (as seen).

3) Deletion — every pass explained (delete 50, which has two children)

Steps:

1. If the node is a leaf node, remove it by removing the link to it.
2. If the node only has one child node, connect the parent node of the node you want to remove to that child node.
3. If the node has both right and left child nodes: Find the node's in-order successor, change values with that node, then delete it.

Deleting a node has 3 cases**:**

1. **Leaf** (no children): remove directly.
2. **One child**: replace node by its child.
3. **Two children**: replace node's key with its **inorder successor** (minimum in right subtree), then delete that successor node.

Let's **delete 50** (root, two children):

Step 1: Find node to delete.

- Compare 50 vs 50 → found at root.

Step 2: Since it has two children, find its inorder successor.

- Inorder successor = **min of right subtree** (rooted at 70).
- Go to 70 → go left to 60 → 60 has no left child → **successor = 60**.

Step 3: Replace.

- Copy successor's key (**60**) into the node being deleted (root).
- Tree keys now *temporarily* look like (value replaced at root):

```
        60
      /    \
    30      70
   /  \      \
 20   40     80
(We'll remove the old 60 next)
```

Step 4: Delete the successor node (the original 60) from the right subtree.

- At node 70, go left to 60; it's a **leaf** → remove it.

Final tree after deletion of 50:

```
        60
      /    \
    30      70
   /  \      \
 20   40     80
```

4) Traversals (display order)

1) Inorder Traversal

- Inorder traversal is one of the **Depth First Traversals** of a tree.
- The recursive rule is:

Inorder(node):
1. **Traverse the left subtree**
2. **Visit the root node**
3. **Traverse the right subtree**

- In a **BST**, inorder traversal **always gives elements in ascending (sorted) order**.

2)What is Postorder Traversal?

- In **Postorder Traversal**, the visiting order is:

Postorder(node):
1. **Traverse the left subtree**
2. **Traverse the right subtree**
3. **Visit the root node**

- In short: **Left → Right → Root**

- **Ascending order** = **Inorder** (Left, Node, Right).

  For the original tree: **20, 30, 40, 50, 60, 70, 80**

- **Descending order** = **Reverse Inorder** (Right, Node, Left).
  For the original tree: **80, 70, 60, 50, 40, 30, 20**

<u>Input:</u> Suppose the following cities and populations are inserted:

City	Population
Pune	50,00,000
Mumbai	1,20,00,000
Delhi	1,80,00,000
Nagpur	30,00,000

<u>Output:</u>
- Ascending order → Delhi → Mumbai → Nagpur → Pune
- Searching for “Nagpur” may take up to 3 comparisons depending on insertion order.

<u>Algorithm/Pseudo code:</u>

1) *Algorithm/Pseudo Code:*

Insert (root, city, population):

1. If root is NULL → create new node.
2. If city < root.city → insert in left subtree.
3. If city > root.city → insert in right subtree.
4. Else → city already exists.

Search (root, city):

1. Initialize comparisons = 0.
2. While root ≠ NULL:
    - comparisons++
    - If city == root.city → return found with comparisons.
    - Else if city < root.city → go to left.
    - Else → go to right.
3. If not found → return "not found" with comparisons.

Delete (root, city):

1. If root is NULL → return.
2. Traverse left/right based on city comparison.
3. If match found:
    - Case 1: No child → delete directly.
    - Case 2: One child → replace with child.
    - Case 3: Two children → replace with inorder successor.

Display Ascending: Inorder(root)
Display Descending: Reverse Inorder(root)

<u>Software required:</u> Open Source Python, Programming tool like Jupyter Notebook, PyCharm, Spyder.

<u>Conclusion:</u> Thus, the students have successfully implemented BST operations for city management, including insertion, deletion, updating, searching (with comparison counts), and displaying cities in ascending/descending order.

```
#include <iostream>
#include <string>
using namespace std;

struct Node {

string city;

int population;

Node* left;

Node* right;


Node(string c, int p) {


city = c;


population = p;


left = right = nullptr;

}
};

class BST {

Node* root;
```

```
Node* insert(Node* node, string city, int population) {

if (node == nullptr) {

cout << "City " << city << " inserted successfully!\n";

return new Node(city, population);

}

if (city < node->city)

node->left = insert(node->left, city, population);

else if (city > node->city)

node->right = insert(node->right, city, population);

else

cout << "City already exists!\n";
```

```
return node;

}

Node* minValueNode(Node* node) {

Node* current = node;

while (current && current->left != nullptr)

current = current->left;

return current;

}

Node* deleteNode(Node* root, string city) {

if (root == nullptr) return root;

if (city < root->city)
```

```
root->left = deleteNode(root->left, city);

else if (city > root->city)

root->right = deleteNode(root->right, city);

else {

// Node with one child or no child

if (root->left == nullptr) {

Node* temp = root->right;

delete root;

return temp;

} else if (root->right == nullptr) {

Node* temp = root->left;
```

```
delete root;

return temp;

}

// Node with two children

Node* temp = minValueNode(root->right);

root->city = temp->city;

root->population = temp->population;

root->right = deleteNode(root->right, temp->city);

}

return root;

}
```

```
void inorder(Node* node) {

if (node != nullptr) {

inorder(node->left);

cout << node->city << " (" << node->population << ")\n";

inorder(node->right);

}

}

void descending(Node* node) {

if (node != nullptr) {

descending(node->right);

cout << node->city << " (" << node->population << ")\n";
```

```
descending(node->left);

}

}

Node* search(Node* node, string city, int &comparisons) {

comparisons++;

if (node == nullptr || node->city == city)

return node;

if (city < node->city)

return search(node->left, city, comparisons);

else

return search(node->right, city, comparisons);
```

```
}

public:

BST() { root = nullptr; }


void addCity(string city, int population) {


root = insert(root, city, population);

}


void deleteCity(string city) {


root = deleteNode(root, city);

}


void updatePopulation(string city, int population) {


int comparisons = 0;


Node* node = search(root, city, comparisons);
```

```
if (node != nullptr) {

node->population = population;

cout << "Population of " << city << " updated successfully!\n";

} else {

cout << "City not found!\n";

}

}

void displayAscending() {

cout << "Cities in Ascending order:\n";

inorder(root);

}
```

```
void displayDescending() {

cout << "Cities in Descending order:\n";

descending(root);

}

void searchCity(string city) {

int comparisons = 0;

Node* node = search(root, city, comparisons);

if (node != nullptr) {

cout << "City found: " << node->city << " (Population: "

<< node->population << ")\n";

} else {
```

```
cout << "City not found!\n";

}

cout << "Comparisons made: " << comparisons << endl;

}
};

int main() {

BST bst;

int choice, population;

string city;

do {

cout << "\n--- City Management Menu ---\n";

cout << "1. Add City\n";

cout << "2. Delete City\n";
```

```
cout << "3. Update Population\n";

cout << "4. Display Ascending Order\n";

cout << "5. Display Descending Order\n";

cout << "6. Search City\n";

cout << "7. Exit\n";

cout << "Enter choice: ";

cin >> choice;


switch (choice) {

case 1:

cout << "Enter city name: ";
```

```
cin >> city;

cout << "Enter population: ";

cin >> population;

bst.addCity(city, population);

break;

case 2:

cout << "Enter city name to delete: ";

cin >> city;

bst.deleteCity(city);

break;

case 3:
```

```
cout << "Enter city name to update: ";

cin >> city;

cout << "Enter new population: ";

cin >> population;

bst.updatePopulation(city, population);

break;

case 4:

bst.displayAscending();

break;

case 5:

bst.displayDescending();
```

```
break;

case 6:

cout << "Enter city name to search: ";

cin >> city;

bst.searchCity(city);

break;

case 7:

cout << "Exiting...\n";

break;

default:

cout << "Invalid choice!\n";
```

```
}

} while (choice != 7);


return 0;
}
```